

```

#version 460
#extension GL_EXT_shader_16bit_storage : require
#extension GL_KHR_shader_subgroup_basic : require
#extension GL_KHR_shader_subgroup_arithmetic : require

// 컴파일 타임 또는 셰이더 레이아웃에서 Wave32 또는 Wave64 선택적 타겟팅 설정
// (예: layout(local_size_x = 32) 또는 64)
layout(local_size_x = 64, local_size_y = 1, local_size_z = 1) in;

// 1. FP32 데이터 스트림 (Native Standard FP32 Path)
layout(set = 0, binding = 0) readonly buffer FP32Input {
    float fp32_in[];
};

// 2. 패킹된 FP16x2 데이터 스트림 (Packed Half Path - 32비트 레지스터 공간에 FP16 두 개
// 적재)
layout(set = 0, binding = 1) readonly buffer FP16PackedInput {
    uint fp16_packed_in[];
};

layout(set = 0, binding = 2) writeonly buffer OutputBuffer {
    float result_out[];
};

void main() {
    uint globalID = gl_GlobalInvocationID.x;

    // Wavefront 크기 (32 또는 64) 확인용 하위 서브그룹 바운드 체크
    uint waveSize = gl_SubgroupSize; // 하드웨어 웨이브 사이즈 (32 또는 64 동적/정적
    바인딩)

    // [Path A] Native FP32 Pipeline (표준 IEEE 754 파이프라인 연산)
    float val_f32 = fp32_in[globalID];
    float computed_f32 = val_f32 * val_f32 + 1.5f; // 단정밀도 독립 연산

    // [Path B] Packed FP16x2 Pipeline (RDNA 하드웨어의 ALU 쪼개기 SIMD 가속)
    // 32비트 레지스터 슬롯 하나에서 FP16 두 개를 언팩하여 동시 연산 수행
    uint packed_val = fp16_packed_in[globalID];
    vec2 unpacked_f16 = unpackHalf2x16(packed_val);

    // SIMD 웨이브 레인 내에서 두 개의 하프 프리시전 연산 병렬 처리
    vec2 computed_f16 = unpacked_f16 * unpacked_f16 + vec2(1.5h);

    // 최종 결과를 표준 FP32 스칼라로 승격 합산하여 출력 스토어
    float final_reduced = computed_f32 + (float(computed_f16.x) + float(computed_f16.y));

    // Wavefront 크기(32/64)별 서브그룹 연산 최적화 분기 모사
    if (waveSize == 32) {

```

```
// Wave32 특화 레인 최적화 패스 (고클럭/저지연 지향)
result_out[globalID] = subgroupAdd(final_reduced) * 0.5f;
} else {
    // Wave64 특화 레인 최적화 패스 (고대역폭/대규모 스루풋 지향)
    result_out[globalID] = subgroupAdd(final_reduced);
}
}
```